

HRC

High-Reliability C++ Coding Standard

Version 1.0

0mWindyBug

0xntpwr

April 2026

A prescriptive coding standard for systems-level C++ — kernel drivers, instrumentation agents, security tooling, and real-time services. This standard enforces memory safety through RAIL, eliminates undefined behavior through initialization and bounds-checking rules, and mandates deterministic resource management to produce code that is auditable, testable, and resistant to exploitation.

Contents

Abstract	4
1 Naming	4
1.1 Variables — M	4
1.2 Functions — M	4
1.3 Types — M	5
1.4 Namespaces — M	5
1.5 Files — M	5
2 Code Organization	5
2.1 Namespace structure — R	5
2.2 Header files — M	6
2.3 Implementation files — M	6
2.4 Include order — M	6
2.5 No circular header dependencies — M	7
3 Modern C++	7
3.1 <code>constexpr</code> for constants — M	7
3.2 <code>nullptr</code> only — M	7
3.3 Range-based loops — R	7
3.4 Structured bindings — REC	7
3.5 <code>auto</code> — use with judgment — R	7
3.6 Smart pointers — M	7
3.7 Move semantics — R	8
3.8 <code>[[nodiscard]]</code> — R	8
3.9 <code>std::string_view</code> for read-only strings — REC	8
4 RAII and Resource Management	8
4.1 Everything through RAII — M	8
4.2 Delete copy on resource owners — M	8
4.3 RAII locks only — M	9
4.4 Custom deleters for OS handles — REC	9
5 Error Handling	9
5.1 <code>std::optional</code> for “maybe no result” — R	9
5.2 <code>bool</code> for simple success/failure — R	9
5.3 Result types for rich errors — REC	9
5.4 Exception policy — R	9
5.5 Never ignore return values — M	10
5.6 Check every OS/API call — M	10
6 Const Correctness	10
6.1 <code>const</code> methods — M	10
6.2 <code>const</code> references for input — M	10
6.3 <code>const</code> variables — R	10
6.4 <code>constexpr</code> over <code>const</code> — REC	10
7 Concurrency	10
7.1 Document thread safety on every shared class — R	10
7.2 Minimize shared mutable state — R	11
7.3 Lock ordering — M	11

7.4	Don't hold locks across external calls — R	11
7.5	<code>std::atomic</code> for simple flags — R	11
8	Memory Safety	11
8.1	No manual <code>new</code> / <code>delete</code> — M	11
8.2	No C-style casts — M	11
8.3	Bounds-check external data — M	12
8.4	Initialize everything — M	12
8.5	Restrict pointer arithmetic — M	12
9	Code Style	12
9.1	Indentation — M	12
9.2	Line length — R	12
9.3	Comments — R	13
9.4	No magic numbers — M	13
10	Platform	13
10.1	Windows macros — M	13
10.2	Library linking — REC	13
10.3	GUIDs — R	13
10.4	Platform abstraction — REC	14
11	Testing and Debugging	14
11.1	Debug output — R	14
11.2	<code>static_assert</code> — R	14
11.3	Runtime assertions — R	14
11.4	Unit tests — R	14
11.5	Sanitizers — REC	14
12	Build and Tooling	14
12.1	Maximum warnings, errors on warnings — M	14
12.2	Static analysis in CI — R	15
12.3	<code>clang-format</code> — M	15
12.4	Reproducible builds — REC	15
13	Documentation	15
13.1	Public APIs — M	15
13.2	<code>ARCHITECTURE.md</code> — R	15
13.3	<code>DECISIONS.md</code> — REC	15
14	Appendix A: Banned Patterns	16
15	Appendix B: File Template	16
16	Appendix C: Pre-Merge Checklist	17

Abstract

This document specifies coding rules for C++17 and later codebases operating in adversarial or safety-critical environments. The standard addresses three primary risk domains: memory safety violations exploitable as security vulnerabilities, resource leaks under exceptional control flow, and concurrency defects that manifest only under production load.

Rules are enforced through compiler warnings treated as errors, static analysis (clang-tidy, MSVC `/analyze`, PVS-Studio), and code review. Each rule carries an explicit severity — Mandatory (M), Required (R), or Recommended (REC).

Severity Definitions:

Tag	Meaning
M	Mandatory. Blocks merge. No exceptions without written waiver.
R	Required. Follow unless a documented technical reason prevents it.
REC	Recommended. Good practice. Not mechanically enforced.

1 Naming

1.1 Variables — M

- **Locals:** `camelCase`
- **Members:** `camelCase_` with trailing underscore
- **Constants:** `kPascalCase`
- **Parameters:** `camelCase`

```
int eventCount = 0;
std::string processName;

class Example {
    std::mutex mutex_;
    EventRouter* eventRouter_;
};

constexpr size_t kMaxFileNameLength = 80;

void ProcessEvent(const Event& event, uint32_t processId);
```

1.2 Functions — M

`PascalCase`. Private helpers go in anonymous namespaces in the `.cpp` file.

```
void StartEventLoop();
bool CanHandle(const GUID& providerId);

namespace {
    bool IsValidString(const std::string& str) { ... }
}
```

1.3 Types — M

Classes and structs: **PascalCase**. Interfaces: **IPascalCase** — the **I** prefix makes inheritance hierarchies immediately scannable. Enums: always **enum class**, **PascalCase** for both name and values. Unscoped enums are banned — they pollute the enclosing namespace and implicitly convert to integers.

```
class EventRouter;
struct IEventDecoder;
struct Event;

enum class ProcessOpcode : USHORT {
    Start = 1,
    Stop = 2,
    DCStart = 11
};
```

1.4 Namespaces — M

All lowercase, **::** separated.

```
namespace project::core { }
namespace project::sinks { }
namespace project::util { }
```

1.5 Files — M

PascalCase.hpp and **PascalCase.cpp**. One class per file, names match. Tightly-coupled helper types may share a file.

2 Code Organization

2.1 Namespace structure — R

Organize by functional area. Each area owns a namespace.

```
project/
├─ core/           # Interfaces and infrastructure
├─ sources/        # Input implementations
└─ sinks/          # Output implementations
```

```
|— decoders/      # Data interpretation
|— util/          # Shared utilities
```

2.2 Header files — M

```
#pragma once

#include <vector>
#include <memory>

#ifndef WIN32_LEAN_AND_MEAN
#define WIN32_LEAN_AND_MEAN
#endif
#include <Windows.h>

#include "Event.hpp"

namespace project::core {

class ClassName {
public:
    // Public interface
private:
    // Members
};

} // namespace project::core
```

2.3 Implementation files — M

```
#include "ClassName.hpp"
#include <algorithm>

namespace project::core {

namespace {
    constexpr int kInternalConstant = 42;
    bool HelperFunction() { ... }
}

// Implementation

} // namespace project::core
```

2.4 Include order — M

Separated by blank lines, in this order:

1. Corresponding header (`.hpp` files)
2. C system headers

3. C++ standard library
4. Platform headers
5. Third-party libraries
6. Project headers

2.5 No circular header dependencies — M

Headers must be self-contained — including one alone compiles without errors. Use forward declarations to break cycles.

3 Modern C++

3.1 `constexpr` for constants — M

No `#define` for numeric or string constants. Ever.

```
constexpr ULONG kBufferSize = 2048;    // correct
#define BUFFER_SIZE 2048                // wrong
```

3.2 `nullptr` only — M

Not `NULL`, not `0`.

3.3 Range-based loops — R

Prefer over index-based. Use `const auto&` when not modifying.

3.4 Structured bindings — REC

Good for map iteration and multi-value returns when it helps readability.

```
for (const auto& [pid, buffer] : processBuffers_) { ... }
```

3.5 `auto` — use with judgment — R

Use when the type is obvious from the right-hand side. Don't use when the reader needs to know the exact type.

```
auto& buffer = processBuffers_[processId];    // obvious
auto ptr = std::make_unique<Decoder>();       // obvious
auto result = Compute(input);                 // what type?
```

3.6 Smart pointers — M

`std::unique_ptr` for exclusive ownership. `std::shared_ptr` for shared. Raw owning pointers are banned. Raw non-owning pointers are fine when the caller guarantees lifetime.

3.7 Move semantics — R

Use `std::move` when transferring ownership. Resource-holding classes should implement move operations. Moved-from objects must be in a valid (if unspecified) state.

3.8 `[[nodiscard]]` — R

Any function whose return value matters for correctness gets `[[nodiscard]]`. This catches “forgot to check the return” bugs at compile time.

```
[[nodiscard]] std::optional<Event> Decode(  
    const EVENT_RECORD& record);  
[[nodiscard]] bool Initialize();
```

3.9 `std::string_view` for read-only strings — REC

Functions that read a string but don’t store it should take `std::string_view`, not `const std::string&`. Avoids allocations when the caller has a literal or substring.

4 RAII and Resource Management

4.1 Everything through RAII — M

Every resource — memory, file handle, socket, mutex, registry key, OS handle — releases through a destructor. Manual `close()` / `CloseHandle()` / `free()` in mainline code is banned. Those calls live inside RAII wrappers.

```
class HandleGuard {  
public:  
    explicit HandleGuard(HANDLE h) : handle_(h) {}  
    ~HandleGuard() {  
        if (handle_ != INVALID_HANDLE_VALUE)  
            CloseHandle(handle_);  
    }  
    HandleGuard(const HandleGuard&) = delete;  
    HandleGuard& operator=(const HandleGuard&) = delete;  
    HANDLE Get() const { return handle_; }  
private:  
    HANDLE handle_;  
};
```

4.2 Delete copy on resource owners — M

If the class owns a unique resource, delete the copy constructor and copy assignment. Double-free is not a theoretical concern.

4.3 RAII locks only — M

Never call `.lock()` / `.unlock()` manually. `std::lock_guard` or `std::scoped_lock`, always. The manual approach breaks on any early return or exception.

```
{
    std::lock_guard<std::mutex> lock(mutex_);
    sharedState_.Update(data);
}
```

4.4 Custom deleters for OS handles — REC

`std::unique_ptr` with custom deleters is cleaner than a full RAII wrapper for simple handle types.

```
auto fileHandle = std::unique_ptr<
    void, decltype(&CloseHandle)>(
    CreateFileW(...), &CloseHandle
);
```

5 Error Handling

5.1 `std::optional` for “maybe no result” — R

When a function can legitimately produce nothing, return `std::optional`. Forces the caller to check.

```
[[nodiscard]] std::optional<Event> Decode(
    const EVENT_RECORD& eventRecord);
```

5.2 `bool` for simple success/failure — R

When the caller doesn’t need to distinguish failure modes.

5.3 Result types for rich errors — REC

When callers need to know *why* something failed, use `std::expected<T, E>` (C++23) or a project-level `Result<T, E>`.

5.4 Exception policy — R

Exceptions for truly exceptional situations: out of memory, corrupted invariants. Not for expected failures, not on hot paths, not in callback-heavy code. If the project uses `-fno-exceptions`, all error handling is through return values.

Document your exception policy at the project level. The worst outcome is half the codebase expecting exceptions and the other half using error codes.

5.5 Never ignore return values — M

Every function returning success/failure, `HRESULT`, `NTSTATUS`, or `std::optional` gets checked. `[[nodiscard]]` enforces this at compile time.

5.6 Check every OS/API call — M

No exceptions to this rule. Every `CreateFile`, every `RegOpenKey`, every `malloc`.

```
HANDLE hFile = CreateFileW(path, ...);
if (hFile == INVALID_HANDLE_VALUE) {
    LogError("CreateFileW failed: %lu",
            GetLastError());
    return std::nullopt;
}
```

6 Const Correctness

6.1 `const` methods — M

If it doesn't modify state, mark it `const`.

6.2 `const` references for input — M

Read-only parameters by `const&` (or `std::string_view` for strings).

6.3 `const` variables — R

If a variable shouldn't change after initialization, say so.

```
const std::string fileName = GetFileName();
const auto startTime =
    std::chrono::steady_clock::now();
```

6.4 `constexpr` over `const` — REC

If a value can be computed at compile time, `constexpr` gives better optimization and earlier error detection.

7 Concurrency

7.1 Document thread safety on every shared class — R

Class-level comment: thread-safe, not thread-safe, or conditionally thread-safe. No ambiguity.

```
/// @brief Routes events to registered sinks.  
/// @threadsafety Not thread-safe. All calls  
/// must originate from the processing thread.  
class EventRouter { ... };
```

7.2 Minimize shared mutable state — R

Prefer message passing, work queues, or immutable data. Every mutex is a potential deadlock, priority inversion, and performance bottleneck.

7.3 Lock ordering — M

Multiple locks? Document the global acquisition order. Violating it causes deadlocks that only reproduce under production load.

7.4 Don't hold locks across external calls — R

No virtual methods, callbacks, or library calls while holding a lock. Copy what you need, release, then call. Prevents an entire category of deadlock and priority inversion bugs.

7.5 `std::atomic` for simple flags — R

Boolean flags and counters shared between threads: `std::atomic`, not a mutex.

```
std::atomic<bool> shutdownRequested_{false};
```

8 Memory Safety

8.1 No manual `new` / `delete` — M

`std::make_unique` and `std::make_shared`. Direct `new` only inside factory functions that immediately wrap the result.

8.2 No C-style casts — M

Use `static_cast`, `reinterpret_cast`, `const_cast`, `dynamic_cast`. C-style casts are invisible to grep, invisible to static analysis, and bypass type safety.

```
// explicit about what's happening  
auto count = static_cast<uint32_t>(rawValue);  
  
// could be anything  
auto count = (uint32_t)rawValue;
```

8.3 Bounds-check external data — M

Data from outside your process — network, files, IPC, shared memory — gets bounds-checked access. Use `std::span`, `std::vector::at()`, or explicit length validation. Raw pointer arithmetic on untrusted input is banned.

8.4 Initialize everything — M

Every variable initialized at declaration. Uninitialized reads are undefined behavior, and compilers exploit UB aggressively. This is a security issue, not a style preference.

```
int count = 0;
HANDLE handle = INVALID_HANDLE_VALUE;
```

8.5 Restrict pointer arithmetic — M

Pointer arithmetic and `reinterpret_cast` limited to documented low-level operations: serialization, hardware registers, OS structure parsing. Each use gets a comment explaining why it is necessary and what invariant guarantees safety. “Needed for cast” or “required here” are not justifications — the comment must state what the memory layout is and why the reinterpretation is valid.

9 Code Style

9.1 Indentation — M

4 spaces. No tabs. Opening brace same line. Closing brace own line.

```
void Function() {
    if (condition) {
        DoSomething();
    } else {
        DoSomethingElse();
    }
}
```

9.2 Line length — R

100 characters preferred, 120 hard limit. Break long parameter lists with alignment.

```
EtwSource::EtwSource(
    core::EventRouter* eventRouter,
    core::DecoderRegistry* decoderRegistry)
: eventRouter_(eventRouter),
  decoderRegistry_(decoderRegistry) {}
```

9.3 Comments — R

Document public APIs with behavior, not implementation. Explain *why*, not *what*.

```
// ImagePath is a full NT path; extract only the
// final component for human-readable filenames.
std::string ExtractProcessName(
    const std::string& imagePath) const;
```

9.4 No magic numbers — M

Only `0`, `1`, and `-1` as literals in logic. Everything else is a named `constexpr`.

```
constexpr int kMaxRetries = 3;
constexpr std::chrono::seconds kTimeout{5};
```

10 Platform

10.1 Windows macros — M

Before any platform include:

```
#ifndef WIN32_LEAN_AND_MEAN
#define WIN32_LEAN_AND_MEAN
#endif
#ifndef NOMINMAX
#define NOMINMAX
#endif
```

10.2 Library linking — REC

```
#pragma comment(lib, "advapi32.lib");
#pragma comment(lib, "tdh.lib");
```

10.3 GUIDs — R

`constexpr` with `k`-prefixed names:

```
constexpr GUID kProcessProviderGuid = {
    0x3d6fa8d0, 0xfe05, 0x11d0,
    { 0x9d, 0xda, 0x00, 0xc0,
      0x4f, 0xd7, 0xba, 0x7c }
};
```

10.4 Platform abstraction — REC

Isolate platform code behind interfaces. Business logic never contains `#ifdef _WIN32`. Platform-specific implementations go in dedicated files selected at build time.

11 Testing and Debugging

11.1 Debug output — R

`std::fprintf(stderr, ...)`. Must be removable via compile flag or log level without touching code.

11.2 `static_assert` — R

Compile-time invariants: struct sizes, alignment, type traits.

```
static_assert(sizeof(wchar_t) == 2,  
    "Wide char must be 2 bytes");  
static_assert(  
    std::is_trivially_copyable_v<Event>);
```

11.3 Runtime assertions — R

`assert()` for invariants during development. No side effects inside assertions — they disappear in release builds.

11.4 Unit tests — R

Non-trivial logic gets tests. Google Test, Catch2, or doctest. Must build and run from the build system in one command.

11.5 Sanitizers — REC

CI should include ASan (`-fsanitize=address`), UBSan (`-fsanitize=undefined`), and TSan (`-fsanitize=thread`) where the platform supports them. These catch bugs that no amount of code review finds.

12 Build and Tooling

12.1 Maximum warnings, errors on warnings — M

- MSVC: `/W4 /WX`
- GCC/Clang: `-Wall -Wextra -Wpedantic -Werror`

Additional recommended flags:

```
-Wshadow -Wnon-virtual-dtor -Wold-style-cast  
-Wcast-align -Wunused -Woverloaded-virtual  
-Wconversion -Wsign-conversion  
-Wnull-dereference -Wdouble-promotion -Wformat=2
```

12.2 Static analysis in CI — R

At least one, run on every PR:

Tool	Platform	Focus
clang-tidy	Cross	Modernization, bugs, style
MSVC <code>/analyze</code>	Windows	Security, correctness
PVS-Studio	Cross	Deep semantic analysis
cppcheck	Cross	UB, leaks

12.3 clang-format — M

All code formatted with a project `.clang-format` config. Enforced in CI. No style arguments in reviews.

12.4 Reproducible builds — REC

Pin compiler versions. No timestamps in binaries. Deterministic link order. Same inputs, bit-identical outputs.

13 Documentation

13.1 Public APIs — M

Every public class, function, and enum has a comment: purpose, parameters, return, thread safety. Comments must add information beyond what the signature already shows — restating parameter names and types is not documentation. If the function is called `Initialize()`, the comment explains what it initializes, what preconditions must hold, and what state changes on success vs failure.

13.2 ARCHITECTURE.md — R

Top-level document: component graph, data flow, ownership model, threading model. A new contributor should understand the system from this file.

13.3 DECISIONS.md — REC

Why choices were made and what was rejected. Saves the team from relitigating the same arguments.

14 Appendix A: Banned Patterns

Pattern	Sev.	Use Instead
<code>new</code> / <code>delete</code>	M	<code>make_unique</code> / <code>make_shared</code>
C-style cast <code>(type)x</code>	M	<code>static_cast<type>(x)</code>
<code>#define</code> for constants	M	<code>constexpr</code>
<code>NULL</code> or <code>0</code> as pointer	M	<code>nullptr</code>
<code>using namespace std;</code> in headers	M	Explicit <code>std::</code>
<code>using namespace</code> at file scope	R	Targeted <code>using std::vector;</code>
Unscoped <code>enum</code>	M	<code>enum class</code>
Raw owning pointers	M	Smart pointers
<code>goto</code>	M	Structured flow or RAII
Manual <code>.lock()</code> / <code>.unlock()</code>	M	<code>lock_guard</code> / <code>scoped_lock</code>
<code>reinterpret_cast</code> without comment	M	Add justification
<code>std::endl</code> on hot paths	R	<code>'\n'</code>
Magic numbers	M	Named <code>constexpr</code>
Uninitialized variables	M	Initialize at declaration
<code>#pragma warning(disable)</code> uncom- mented	R	Document suppression

15 Appendix B: File Template

```
#pragma once

// ClassName.hpp – What this class does.

#include <cstdint>
#include <memory>
#include <optional>
#include <string>

#include "project/core/IInterface.hpp"

namespace project::component {
```

```
/// @brief What this class does.
/// @threadsafety Thread-safety guarantees.
class ClassName final : public IInterface {
public:
    explicit ClassName(
        std::unique_ptr<Dependency> dep);
    ~ClassName() override;

    ClassName(const ClassName&) = delete;
    ClassName& operator=(const ClassName&) = delete;
    ClassName(ClassName&&) noexcept = default;
    ClassName& operator=(ClassName&&) noexcept
        = default;

    [[nodiscard]] bool Initialize() override;
    [[nodiscard]] std::optional<Result> Process(
        const Input& input) override;

private:
    std::unique_ptr<Dependency> dep_;
};

} // namespace project::component
```

16 Appendix C: Pre-Merge Checklist

1. Zero warnings with `/W4 /WX` or `-Wall -Wextra -Wpedantic -Werror`
2. `clang-format` reports no diff
3. `clang-tidy` clean on changed files
4. Static analyzer: no new findings
5. All tests pass
6. No raw `new` / `delete`
7. No uninitialized variables
8. No C-style casts
9. Public APIs documented
10. `DECISIONS.md` updated if architecture changed